# LogHelper Performance Improvement

Thread-Safe Initialization Strategies for JVM

## Goal of Task

### One Instance

Guarantee exactly one object per JVM instance, ensuring a single point of access throughout the application lifecycle.

### Thread Safety

Thread-safe access under high concurrency with multiple threads attempting simultaneous instantiation.

### Safe Initialization

Zero risk of partial initialization or race conditions that could corrupt the singleton state.

### Performance

Minimal performance overhead while maintaining thread safety and correctness guarantees.

## Approach 1: Synchronized

### Synchronized getInstance()

JVM enforces mutual exclusion on every invocation, ensuring only one thread can enter the method at a time with guaranteed visibility.

1. **synchronized** keyword applied to getInstance()
2. JVM acquires monitor lock on the class object
3. Check if `instance == null`
4. Create new instance via `new LogHelper()`
5. Assign reference and release lock

**Thread Behavior**

Thread A: Acquires lock, creates instance
Thread B: BLOCKED - waiting for lock
Thread C: BLOCKED - waiting for lock
Thread D: BLOCKED - waiting for lock

**Pros**

- ✓ Guarantees exactly one object per JVM
- ✓ Guarantees visibility & atomicity
- ✓ Simple and straightforward
- ✓ No complex state management

**Cons**

- ✗ Lock acquired on **every call**
- ✗ High contention under load
- ✗ Performance bottleneck
- ✗ Poor scalability

## Approach 2: Double-Checked Locking (Volatile + Synchronized)

### Double-Checked Locking Pattern

First check avoids locking after initialization. Synchronization occurs only during creation, with volatile ensuring safe publication.

1. **First check:** `if (instance == null)` - no lock needed
2. Acquire `synchronized` lock on class object
3. **Second check:** `if (instance == null)` inside synchronized block
4. Create instance: `new LogHelper()`
5. Volatile write ensures safe publication to all threads
6. Lock released, subsequent calls skip sync block

**Thread Behavior (After Initialization)**

Thread A: Creates instance inside synchronized block
Thread B: Briefly waits, then sees non-null instance → proceeds
Thread C: First check sees non-null → proceeds immediately (NO LOCK)
Thread D: First check sees non-null → proceeds immediately (NO LOCK)

**Pros**

- ✓ Locking occurs **once per JVM**
- ✓ Near-zero overhead after initialization
- ✓ Excellent scalability under load
- ✓ Production-ready approach

**Cons**

- ✗ More complex implementation
- ✗ Requires understanding of volatile semantics
- ✗ Difficult to get right initially

| Dimension | Synchronized Only | Synchronized + Volatile |
|---|---|---|
| **Correctness** | ✓ | ✓ |
| **Guarantees One Object** | ✓ | ✓ |
| **Lock Frequency** | **Every call** | **Once per JVM** |
| **Performance** | Medium–Low | **High** |
| **Scalability** | Limited | **Excellent** |
| **Implementation Complexity** | Low | Medium |